

ARDUINO MICROCONTROLLER CONCEPTS (CSE 4820)

7th Semester, B.Tech

ASSIGNMENT-1

1. What is the purpose of the setup() and loop() function?

- **setup()**: It is executed **once** when the Arduino program starts. It is used for **initialization**, such as setting pin modes and starting serial communication.
- **loop()**: It is executed **repeatedly** after setup(). It contains the **main program instructions** that the Arduino continuously performs.

Key Point: setup() = **initialization once**, loop() = **main program repeated continuously**.

2. What is the purpose of digitalWrite() and delay()? Explain with example.

- **digitalWrite()**: It is used to set a digital pin to either **HIGH (ON)** or **LOW (OFF)**.
- **delay()**: It is used to **pause the program for a specified time in milliseconds (ms)**. For example, delay(1000) pauses the program for **1 second**.

Example:

```
void setup() {  
  pinMode(8, OUTPUT);  
}
```

```
void loop() {  
  digitalWrite(8, HIGH); // Turn ON LED  
  delay(1000);           // Wait for 1 second  
  digitalWrite(8, LOW);  // Turn OFF LED  
  delay(1000);           // Wait for 1 second  
}
```

Key Point: digitalWrite() controls the **ON/OFF state** of a digital pin, while delay() provides a **time delay**.

3. Write the basic structure of an Arduino sketch.

The basic structure of an Arduino sketch consists of **two main functions: setup() and loop()**.

```
// Global declarations
```

```
void setup() {  
    // Initialization statements  
}
```

```
void loop() {  
    // Main program statements  
}
```

- **setup()** executes **once** and is used for initialization.
- **loop()** executes **repeatedly** and contains the main program logic.

Key Point: Every Arduino sketch normally contains the **setup()** and **loop()** functions.

4. What is variable scope? Explain local and global variables with example.

Variable scope is the **region of a program where a variable can be declared and accessed**.

- **Local Variable:** Declared inside a function or block and can be accessed **only within that function or block**.
- **Global Variable:** Declared **outside all functions** and can be accessed by **any function** in the program.

Example:

```
int T = 10;    // Global variable
```

```
void setup() {  
    int x = 5;    // Local variable  
}
```

```
void loop() {
```

```
int y = 20;    // Local variable

T = T + 1;    // Global variable can be accessed
}
```

Key Point: Local variables have **limited scope**, whereas global variables have **program-wide scope**.

5. Write all the data types available in Arduino C++.

The data types available in Arduino C++ include:

void, boolean, char, unsigned char, byte, int, unsigned int, word, long, unsigned long, short, float, double, array, String (char array), and String (object).

Key Point: These data types are used to store different kinds of values such as **integers, decimal values, characters, Boolean values, and strings**.

6. Write all the delay functions available in Arduino C++.

Arduino provides **two delay functions**:

1. **delay()** – Creates a delay in **milliseconds**.
2. **delayMicroseconds()** – Creates a delay in **microseconds**.

Example:

```
delay(1000);        // Delay of 1 second

delayMicroseconds(1000); // Delay of 1000 microseconds
```

Key Point: **delay()** is used for **millisecond delays**, while **delayMicroseconds()** is used for **microsecond delays**.

7. What is an Arduino String? Name any two string functions.

An **Arduino String** is a **String object used for storing and manipulating text**. It uses an uppercase **S**.

Any two String functions are:

1. **length()** – Returns the number of characters in a string.
2. **concat()** – Combines two strings.

Example:

```
String text = "Hello";  
text.concat(" Arduino");
```

Key Point: Arduino String provides methods for **storing, manipulating, and processing text**.

8. Write a program to blink an LED.

```
int ledPin = 13;  
  
void setup() {  
  pinMode(ledPin, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(ledPin, HIGH); // LED ON  
  delay(1000);                // Wait 1 second  
  
  digitalWrite(ledPin, LOW);  // LED OFF  
  delay(1000);                // Wait 1 second  
}
```

Explanation: digitalWrite() controls the LED's **ON/OFF state**, while delay(1000) provides a **1-second delay** between the states.

9. Write a program to find the sum and average of 10 random numbers.

```
long sum = 0;  
float average;  
  
void setup() {  
  Serial.begin(9600);
```

```
for (int i = 0; i < 10; i++) {  
    int num = random(1, 101); // Generate random number from 1 to 100  
    sum = sum + num;  
}
```

```
average = sum / 10.0;
```

```
Serial.print("Sum = ");
```

```
Serial.println(sum);
```

```
Serial.print("Average = ");
```

```
Serial.println(average);
```

```
}
```

```
void loop() {
```

```
}
```

Explanation: random(1, 101) generates random numbers from **1 to 100**. The 10 numbers are added to obtain the **sum**, and the sum is divided by **10** to obtain the **average**.

10. Explain and demonstrate the min() and max() functions. Write a code fragment that reads two sensor values and prints both the smaller and the larger of the two.

- **min(x, y):** Returns the **smaller value** between x and y.
- **max(x, y):** Returns the **larger value** between x and y.

Code Fragment:

```
int sensor1, sensor2;
```

```
int smaller, larger;
```

```
void setup() {
```

```
Serial.begin(9600);  
  
}  
  
void loop() {  
    sensor1 = analogRead(A0);  
    sensor2 = analogRead(A1);  
  
    smaller = min(sensor1, sensor2);  
    larger = max(sensor1, sensor2);  
  
    Serial.print("Smaller = ");  
    Serial.println(smaller);  
  
    Serial.print("Larger = ");  
    Serial.println(larger);  
  
    delay(1000);  
}
```

Key Point: min() gives the **smaller sensor value**, while max() gives the **larger sensor value**.

11. Two analog sensors provide readings through A0 and A1. Write an Arduino code fragment using abs() to calculate the difference between the readings and turn ON an LED if their difference is greater than 20.

The **abs() function** returns the **absolute value** of a number. It can be used to find the positive difference between two sensor readings.

```
int sensor1, sensor2;  
  
int difference;  
  
void setup() {
```

```

pinMode(13, OUTPUT);
}

void loop() {
    sensor1 = analogRead(A0);
    sensor2 = analogRead(A1);

    difference = abs(sensor1 - sensor2);

    if (difference > 20) {
        digitalWrite(13, HIGH); // LED ON
    } else {
        digitalWrite(13, LOW); // LED OFF
    }
}

```

Key Point: `abs(sensor1 - sensor2)` gives the **absolute difference** between the two sensor readings. If the difference is **greater than 20**, the LED is turned **ON**.

12. Design an Arduino code fragment that processes a counter from 1 to 20 and turns an LED ON only when the counter reaches a multiple of 5. Use the modulo (%) operator and explain how it identifies the required values.

The **modulo (%) operator** returns the **remainder** after division. A number is a multiple of 5 when its remainder after division by 5 is **zero**.

```

int ledPin = 13;

void setup() {
    pinMode(ledPin, OUTPUT);

    for (int counter = 1; counter <= 20; counter++) {

```

```

if (counter % 5 == 0) {
    digitalWrite(ledPin, HIGH); // LED ON
} else {
    digitalWrite(ledPin, LOW); // LED OFF
}

delay(500);
}
}

void loop() {
}

```

Explanation: The condition `counter % 5 == 0` becomes true for **5, 10, 15, and 20**, because these values leave a remainder of **0** when divided by 5. Therefore, the LED is turned **ON only for these values**.

13. Write an Arduino sketch that gradually increases the brightness of an LED from 0 to 10 and then decreases it back to 0. Use ++ to increase the brightness and -- to decrease it. Display each brightness value on the Serial Monitor.

```

int ledPin = 9;

void setup() {
    pinMode(ledPin, OUTPUT);
    Serial.begin(9600);
}

void loop() {

```

```
// Increase brightness from 0 to 10
for (int brightness = 0; brightness <= 10; brightness++) {
    analogWrite(ledPin, brightness);
    Serial.println(brightness);
    delay(100);
}

// Decrease brightness from 10 to 0
for (int brightness = 10; brightness >= 0; brightness--) {
    analogWrite(ledPin, brightness);
    Serial.println(brightness);
    delay(100);
}
}
```

Explanation:

- brightness++ increases the brightness value by **1**.
- brightness-- decreases the brightness value by **1**.
- Serial.println() displays each brightness value on the **Serial Monitor**.
- The LED brightness varies from **0 → 10 → 0** continuously.

14. Why Does Overflow Occur? How can we Correct Overflow Handling? Explain through the example.

Overflow occurs when a variable is assigned a value **greater than the maximum value that its data type can store**. The value then wraps around to the beginning of its range.

For example, an Arduino UNO unsigned int can store values from **0 to 65,535**.

```
unsigned int x = 65535;
```

```
x = x + 1;
```

```
Serial.println(x);
```

After exceeding **65,535**, the value wraps around to **0**.

Correct Overflow Handling:

- Use a **larger data type** such as unsigned long when a larger range is required.
- Use the appropriate data type for values and calculations that may exceed the variable's range.

Key Point: Overflow occurs when the **maximum storage limit is exceeded**; it can be handled by choosing an **appropriate data type and range**.

15. Write a program to generate 20 random numbers from 0 to 30.

```
void setup() {  
  Serial.begin(9600);  
  
  for (int i = 0; i < 20; i++) {  
    int num = random(0, 31);  
    Serial.println(num);  
  }  
}  
  
void loop() {  
}
```

Explanation:

- random(0, 31) generates a random integer from **0 to 30**.
 - The for loop executes **20 times**, generating 20 random numbers.
 - Serial.println() displays the numbers on the **Serial Monitor**.
-